

BIO-INSPIRED ARTIFICIAL INTELLIGENCE

Genetic Programming

Dr. Giovanni Iacca

giovanni.iacca@unitn.it



UNIVERSITY OF TRENTO - Italy

**Information Engineering
and Computer Science Department**

Genetic Programming



GENETIC PROGRAMMING

THE CHALLENGE

- Virtually all problems in computer science can be recast as a **search for a computer program** that solves the problem. Evolutionary search provides a way to successfully find a computer program in the space of computer programs (seminal hypothesis by Alan Turing, 1948)
- *“How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what is needed to be done, without being told exactly how to do it?”* (attributed to Arthur Samuel, 1959, AI pioneer and coiner of “machine learning”)
- Genetic programming addresses this challenge by providing a method for automatically creating a working computer program from a high-level problem statement of the problem
→ **Automatic programming** (a.k.a. **program synthesis** or **program induction**)



GENETIC PROGRAMMING

GENERALITIES

- A specialization of GAs, where (in most cases) tree-based representations are used
- Original goal: devise a general framework to evolve computer programs
- Currently, used mostly for black-box (data-driven) modeling, forecasting, and classification, as an alternative to Neural Networks and alike
- History:
 - Early attempts to evolve computer programs (not yet as in modern GP): Smith (1980)
 - First GP implementations: Cammer (1985) and Koza (1989), in LISP
 - However, Koza is unanimously considered the “founding father” of GP
- Strength: produces a “human-readable” model/program (differently from Neural Networks). It allows reuse of existing subprograms (facilitates scalability and hierarchical expansion)
- Weaknesses: needs large populations, slow convergence (but, research has made many progresses!)

GENETIC PROGRAMMING

WHY TREES?

Trees are a universal form, e.g. consider:

$y = x * \ln(a) + \sin(z) / \exp(-x) - 3.4$

Arithmetic expression

$(x_1 \text{ AND NOT } x_2) \text{ OR } (\text{NOT } x_1 \text{ AND } x_2)$

Boolean expression

```
i = 1;  
while (i < 20)  
{  
    i = i + 1;  
}
```

Program

"It is difficult, unnatural, and overly restrictive to attempt to represent hierarchies of dynamically varying size and shape with fixed length character strings. For many problems in machine learning and artificial intelligence, the most natural representation for a solution is a computer program."

[Koza, 1994]

IMPLICATIONS

- Adaptive individuals
 - Size of individuals is not fixed
 - Size depends on depth of tree and branching factor
- Domain-specific grammar
 - A grammar needs to be defined, in order to accurately reflect the problem to be solved
 - It should be possible to represent any possible expression using the defined grammar

GENETIC PROGRAMMING

GRAMMAR DEFINITION

- **Terminal set ($\mathbf{T}$):** specifies all the variables and constants, i.e., the leaves of the trees
- **Function set ($\mathbf{F}$):** contains all the functions that can be applied to the elements of the terminal set (arithmetic, Boolean, etc., plus decision structures such as *if-then-else* and loops)
 - Each function f in $\mathbf{F}$ has its own arity (number of arguments)
 - In general, each function f in $\mathbf{F}$ can take any g in $\mathbf{F}$ as argument (closure property $\rightarrow$ GP expressions are usually not typed)
- Correct expressions generated by the grammar can be defined recursively:
 - Every t in $\mathbf{T}$ is a correct expression
 - $f(e_1, e_2, \dots, e_n)$ is a correct expression if and only if:
 - 1) f is in $\mathbf{F}$
 - 2) $\text{arity}(f)=n$ is correct
 - 3) $e_1, e_2, \dots, e_n$ are correct expressions
- **Semantic rules** can be introduced to characterize valid expressions

GENETIC PROGRAMMING

EXAMPLE 1: BOOLEAN EXPRESSION

- Evolve an expression such as:
 $(x_1 \text{ AND NOT } x_2) \text{ OR } (\text{NOT } x_1 \text{ AND } x_2)$

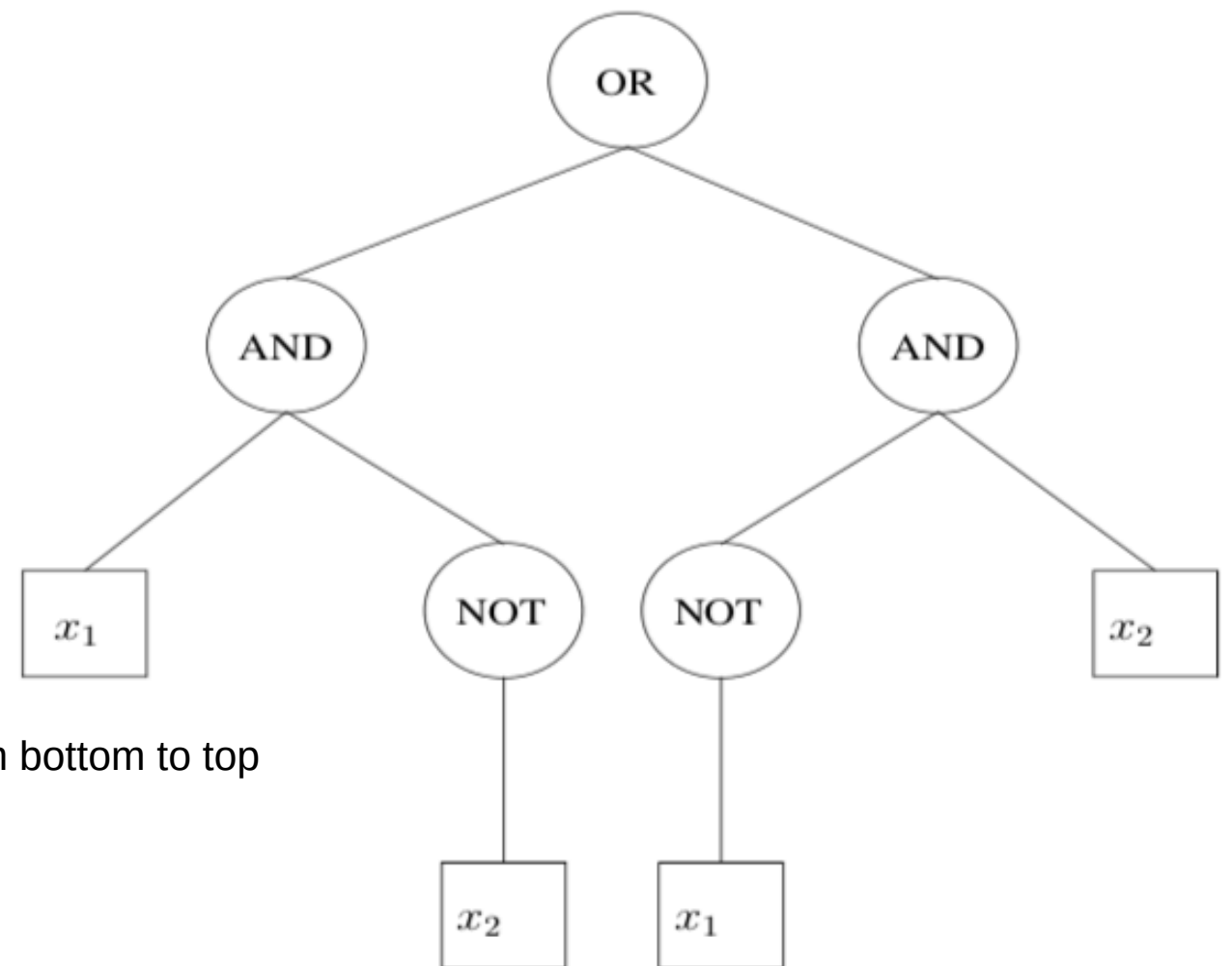
- Given the data set:

x_1	x_2	Target output
0	0	0
0	1	1
1	0	1
1	1	0

this is a training set

input output

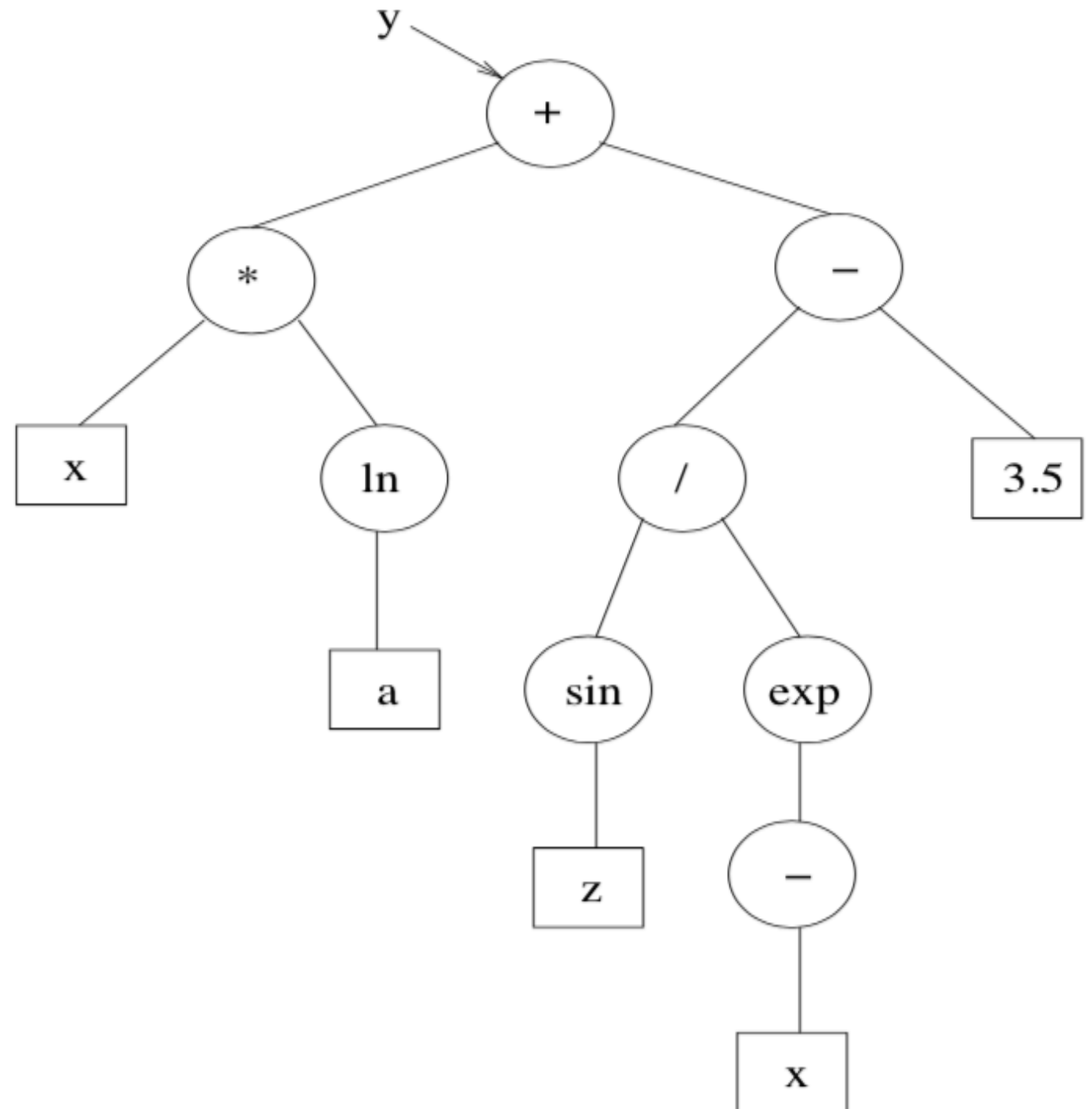
- Function set:
 $\{\text{AND}, \text{OR}, \text{NOT}\}$
- Terminal set:
 $\{x_1, x_2\}$ where $x_1, x_2 \in \{0, 1\}$



GENETIC PROGRAMMING

EXAMPLE 2: SYMBOLIC REGRESSION (WITH GENERATOR FUNCTION)

- Evolve an arithmetic expression such as:
 $y = x * \ln(a) + \sin(z) / \exp(-x) - 3.4$
- Given a set of training in samples in $\mathbf{R}^2$:
 $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Function set:
 $\{-, +, *, /, \sin, \exp, \ln\}$
- Terminal set:
 $\{a, x, z, 3.4\}$, with $a, x, z \in \mathbf{R}$



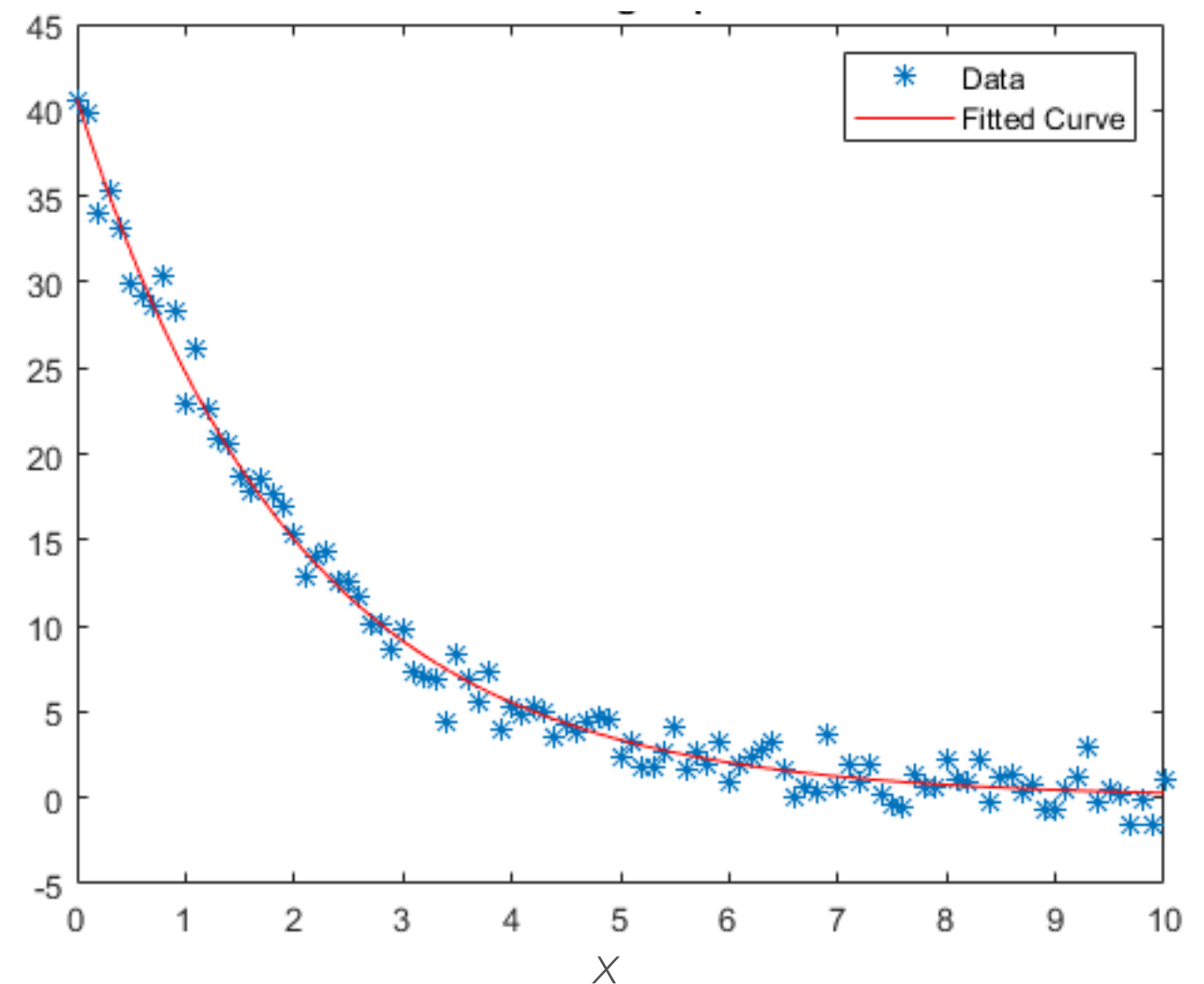
GENETIC PROGRAMMING

EXAMPLE 2: SYMBOLIC REGRESSION (BLACK-BOX)

- Given some points in $\mathbf{R}^2$: $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Find a function $f(x)$ s.t. for every i in $\{1, 2, \dots, n\}$: $f(x_i) = y_i$
- Possible GP solution:
 - Function set: $\{-, +, *, /, \sin, \exp, \ln\}$
 - Terminal set: $\mathbf{R} + \{x\}$
 - Fitness is the sum of squared errors:

$$err(f) = \sum_{i=1}^n (f(x_i) - y_i)^2$$

(or a similar metric such as MSE or RMSE)



GENETIC PROGRAMMING

EXAMPLE 3: COMPUTER PROGRAM

- Evolve the following loop cycle:

```
i=1;  
while (i<20)  
{  
  i=i+1;  
}
```

- Given an input, and a desired program output.

In this (trivial) case:

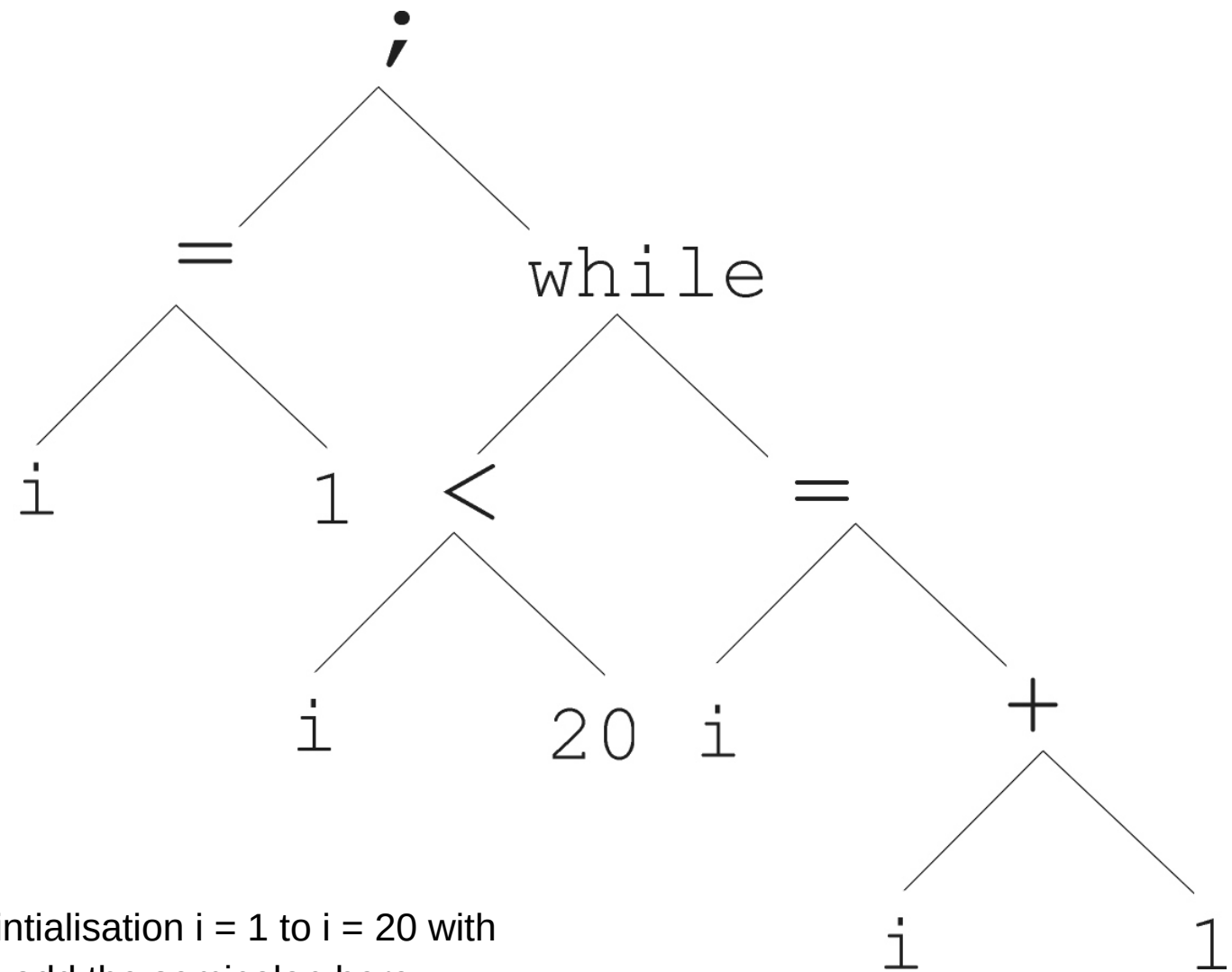
- input: $i=1$
- output: $i=20$

given this

- Function set:
 $\{\text{while}, <, >, =\}$

it will figure how to go from the intialisation $i = 1$ to $i = 20$ with this function set + (we can also add the semicolon here

- Terminal set:
 $\{i, 20\}$, with $i \in \mathbf{Z}^+$

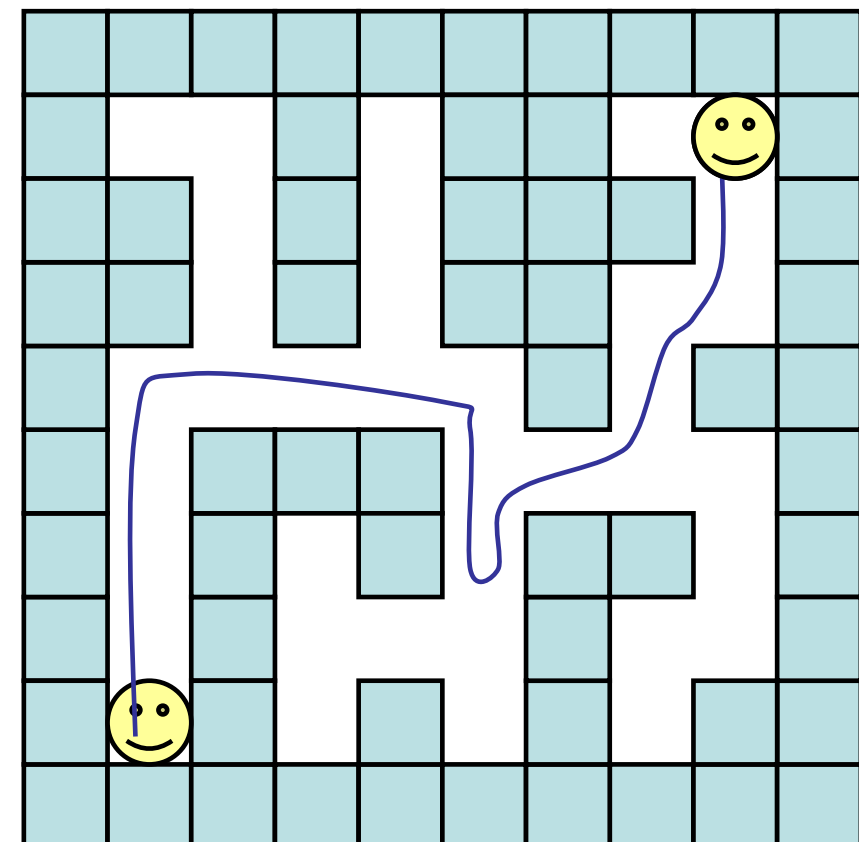


GENETIC PROGRAMMING

EXAMPLE 3: COMPUTER PROGRAM (CONT'D)

- Suppose we want to evolve a controller for guiding an agent (e.g. a mouse) towards a goal (cheese) in a maze
- Function set:
{If-Way-Ahead-Blocked, While-Not-At-Cheese (*root node*)}
- Terminal set:
{Move-Forward, Turn-Left, Turn-Right}

```
While not at the cheese
  If the way ahead is blocked
    Turn right 90 degrees
    Move forward one space
    Move forward one space
    Move forward one space
  Otherwise
    Move forward one space
    Turn right 90 degrees
    Move forward one space
    Move forward one space
    Turn left 90 degrees
    If the way ahead is blocked
      Turn left 90 degrees
    Otherwise
      Move forward one space
```



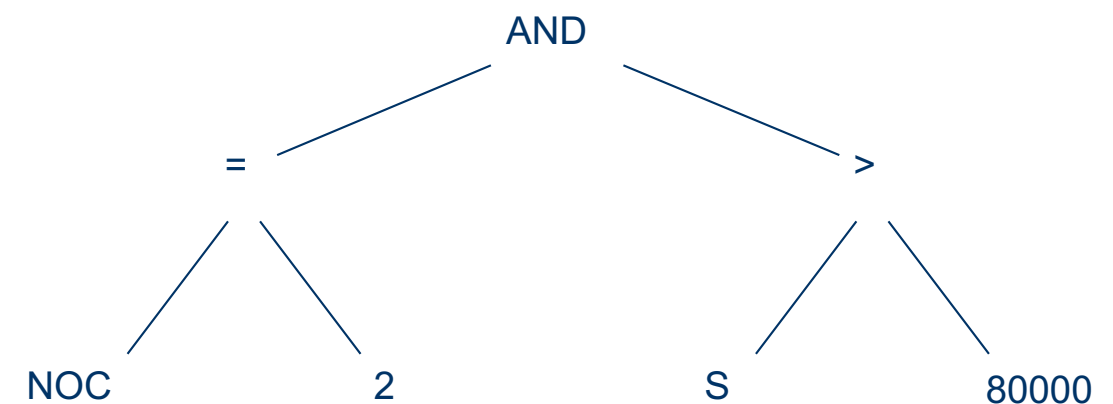
GENETIC PROGRAMMING

EXAMPLE 4: CLASSIFICATION (BANK CREDIT SCORING)

- A bank wants to distinguish “good” from “bad” loan applicants
- A model is needed that matches historical data:

ID	No of Children	Salary	Marital status	Class
ID-1	2	45000	Married	bad
ID-2	0	30000	Single	good
ID-3	1	40000	Divorced	good
...				

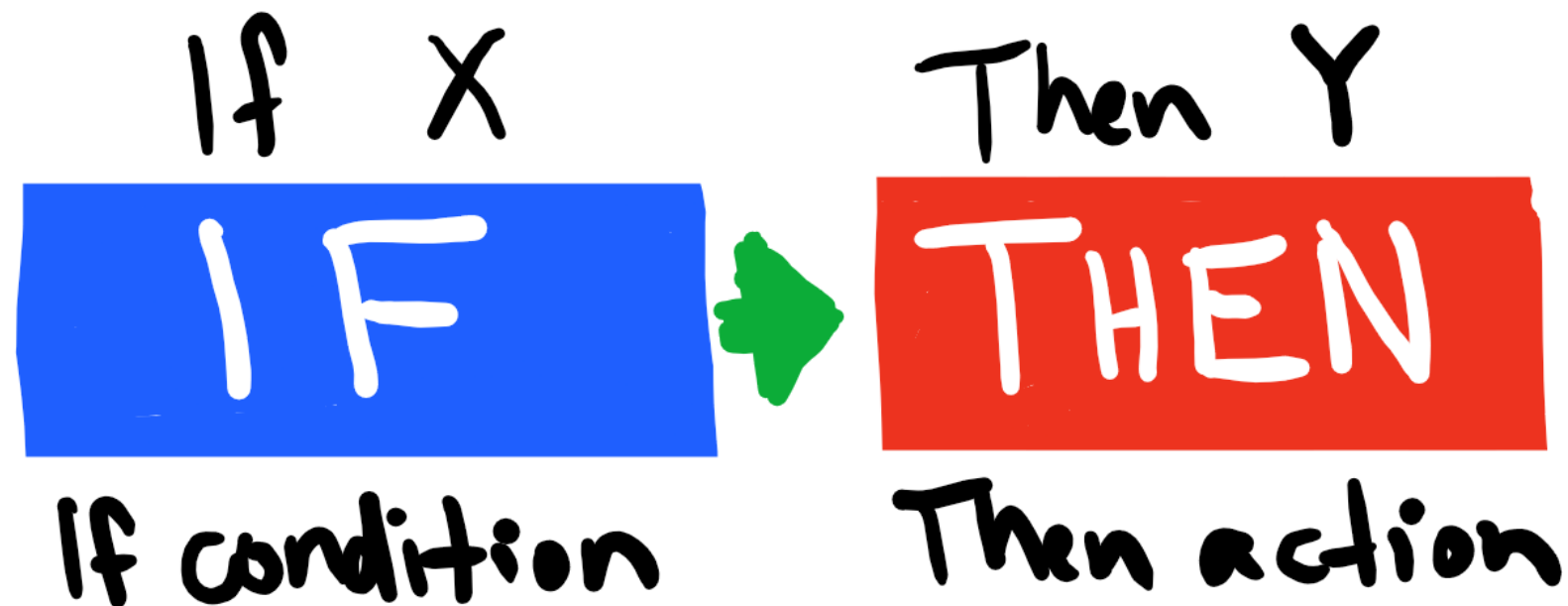
- A possible model:
IF (NOC = 2) AND (S > 80000) THEN good ELSE bad
- Function set:
{AND, OR, NOT, <, >, =}
- Terminal set:
{NOC, S, Married/Single/Divorced, C}, with $C \in \mathbf{Z}^+$



GENETIC PROGRAMMING

EXAMPLE 4: CLASSIFICATION (GENERALIZED)

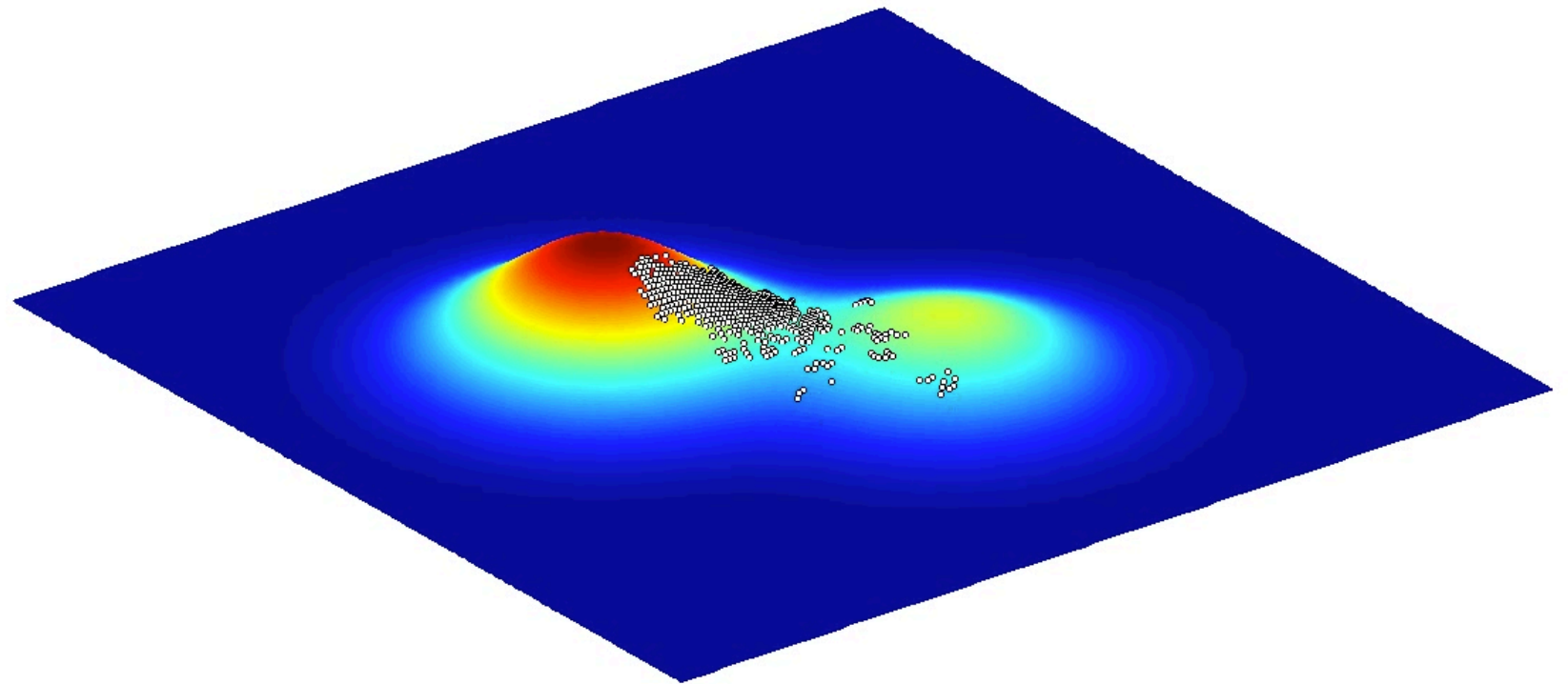
- In general, we want to identify an if-then rule of this kind:
IF [expression] THEN [target output]
- Only unknown is the right expression, hence:
 - Search space is a set of expressions (phenotypes)
 - Fitness of an expression: percentage of well classified cases (supervised learning)
 - Representation (genotypes) of expressions: parse trees, also called S(ymbolic)-Expressions



GENETIC PROGRAMMING

FITNESS FUNCTIONS USED IN GP

- Fitness function is usually problem-dependent
- Fitness calculation requires the expression/program to be evaluated against a number of test cases in order to determine if it matches the desired outcome corresponding to each case:
 - For Boolean expressions: fitness is the number of correctly predicted target outputs
 - For arithmetic expressions: fitness is the sum of squared errors w.r.t. supplied target outputs
 - For computer programs: fitness is the number of correctly generated target outputs
- Fitness function may include a penalty to penalize individuals with undesirable structural properties:
 - Penalize invalid expressions
 - Penalize big tree sizes

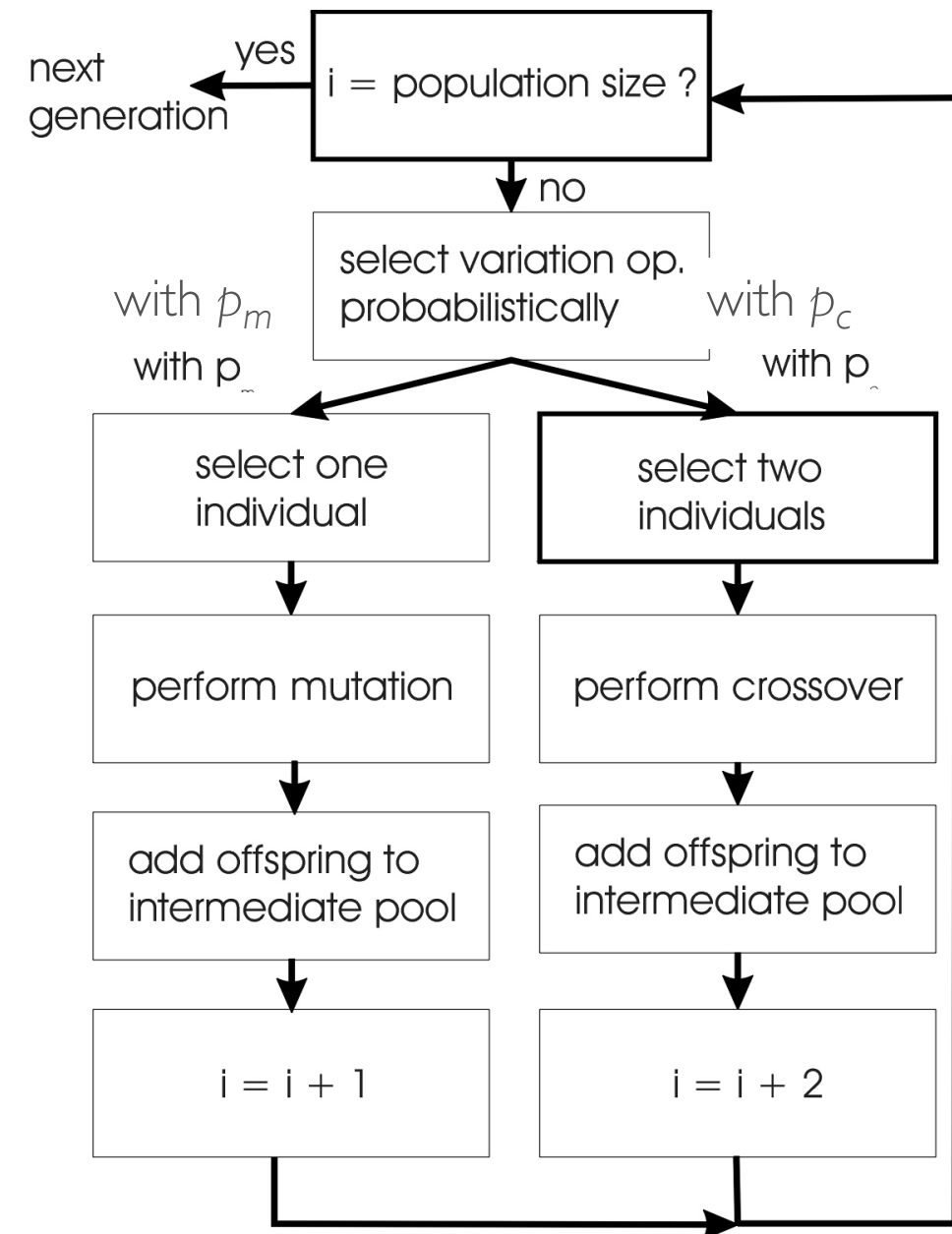


GENETIC PROGRAMMING

DIFFERENCES BETWEEN GP AND GA

- Representation
 - In GA, chromosomes are linear structures (bit strings, integer string, real-valued vectors, permutations)
 - In GP, tree-shaped chromosomes are non-linear structures
- Chromosome size
 - In GA, the size of the chromosomes is fixed
 - In GP, trees may vary in depth and width
- Use of operators (in general):
 - GA uses crossover AND mutation
 - GP uses crossover OR mutation

In both cases crossover and mutation are depending on a given (user-defined) probability.
- In GP, usually crossover is set as highly probable ($p_c > 0.8$), while mutation is set as quite rare ($p_m < 0.05$)



GENETIC PROGRAMMING

ALGORITHMIC DETAILS: INITIAL POPULATION

- Initial trees are randomly generated within a maximum depth (usually, initial trees are small), and considering semantic restrictions expressed by the given grammar
- For each individual, a root is randomly selected from the function set **F**
- Then, two main methods can be used for choosing the nodes below the root:
 - 1) **Full method** (each branch has depth exactly equal to D_{max}):
 - nodes at depth $d < D_{max}$ randomly chosen from function set **F**
 - nodes at depth $d = D_{max}$ randomly chosen from terminal set **T**
 - 2) **Grow method** (each branch has depth at most equal to D_{max}):
 - nodes at depth $d < D_{max}$ randomly chosen from **F** $\cup$ **T**
 - nodes at depth $d = D_{max}$ randomly chosen from **T**
- Usually, full and grow methods are combined into what is called “**ramped half-and-half method**”, i.e. both methods are used, each delivering 50% of the initial individuals

GENETIC PROGRAMMING

ALGORITHMIC DETAILS: PARENT SELECTION

- **Parent selection:**

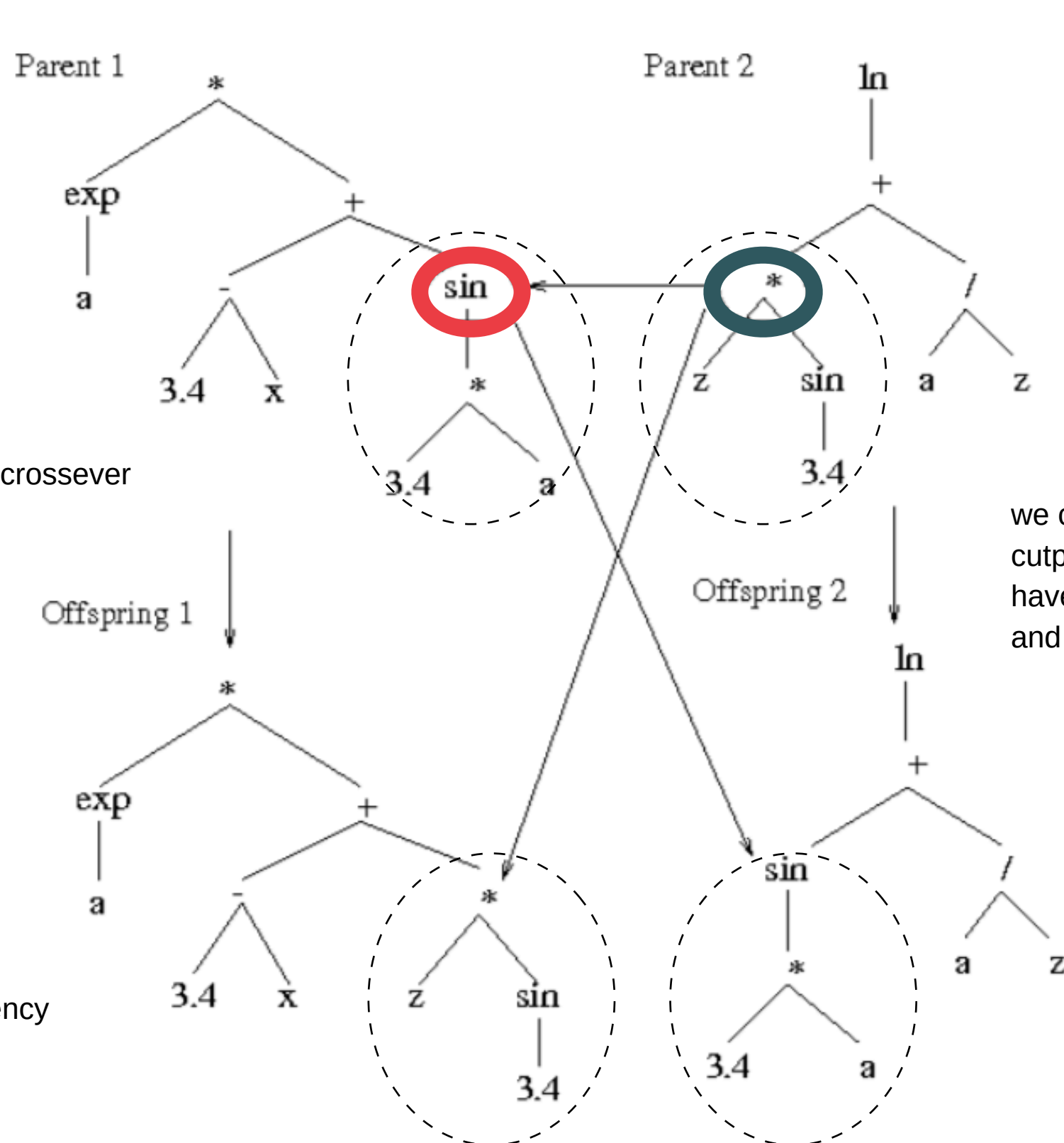
- Classic GP: fitness-proportionate
 - ▶ Remember: fitness is a percentage of correct outputs (should be maximized) or a cumulative error (should be minimized)
- Modern GP implementations: “over-selection” (to increase efficiency)
 - ▶ rank population by fitness, and divide it into two groups:
 - ▶ group 1: best $x\%$ of population, group 2 other $(100-x)\%$
 - ▶ 80% of selection operations chooses from group 1, 20% from group 2
 - ▶ for pop. size = 1000, 2000, 4000, 8000, respectively $x = 32\%, 16\%, 8\%, 4\%$
 - ▶ $x\%$ thresholds are set by rules of thumb

- **Survivor selection:**

- Classic GP: very large populations (50k-600k individuals), with purely generational scheme (thus, no survival of parents)
- Modern GP: smaller populations ($\ll 10k$), with steady-state scheme/elitism

GENETIC PROGRAMMING

ALGORITHMIC DETAILS: CROSSOVER



we decide a cutpoint in the tree and swap the at both cutpoint

whenever i do a mutation and crossever
We have to take into account
that the type is correct

we can instead of choosing one
cutpointin each parent, we can
have n cutpoint in each paren
and swap the 2N cutpoint

but we can have some inconstency
in the operator or a problem
with the type of the variable

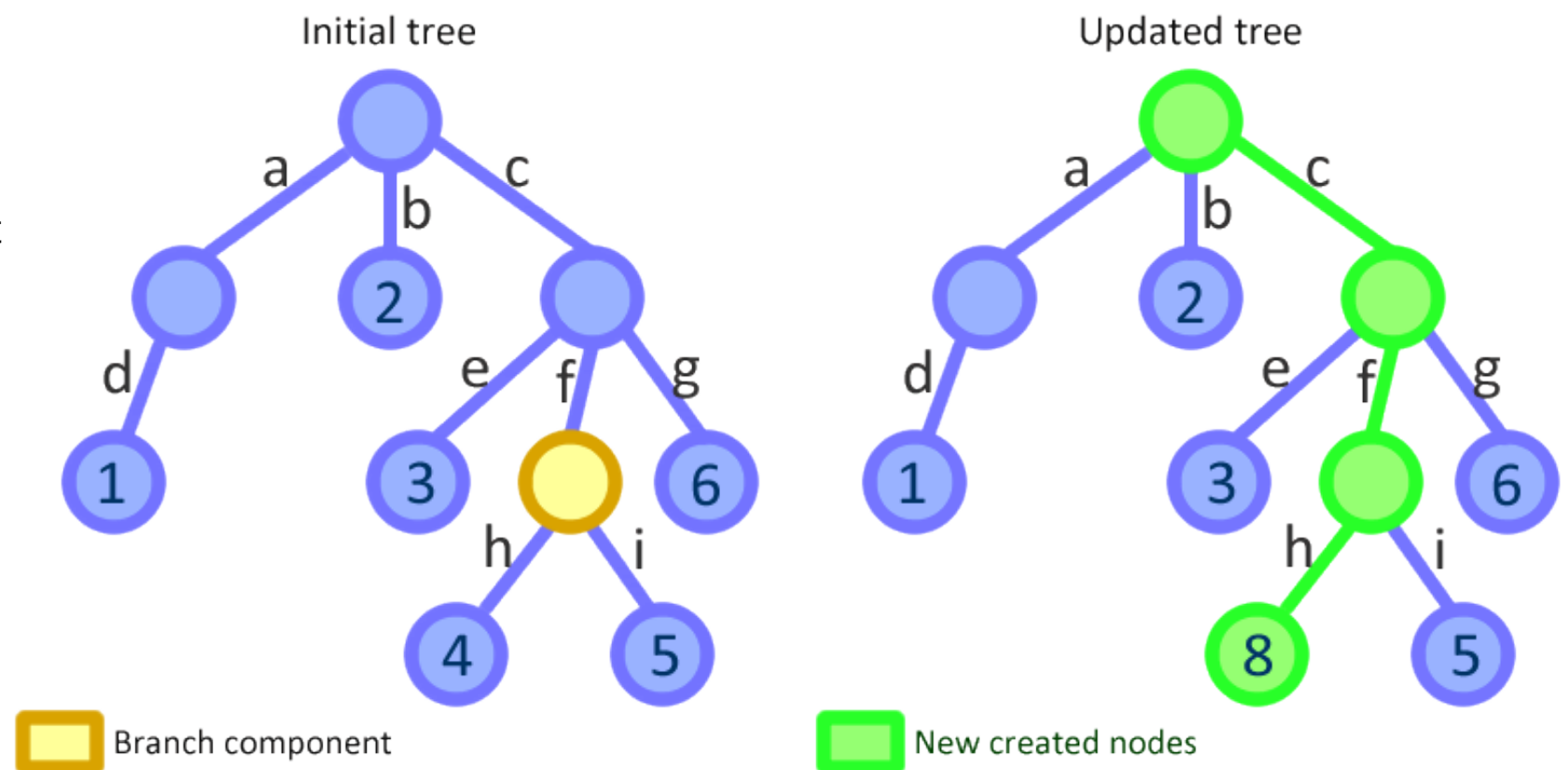
GENETIC PROGRAMMING

ALGORITHMIC DETAILS: MUTATION

The tree-based representation allows a number of possible mutations:

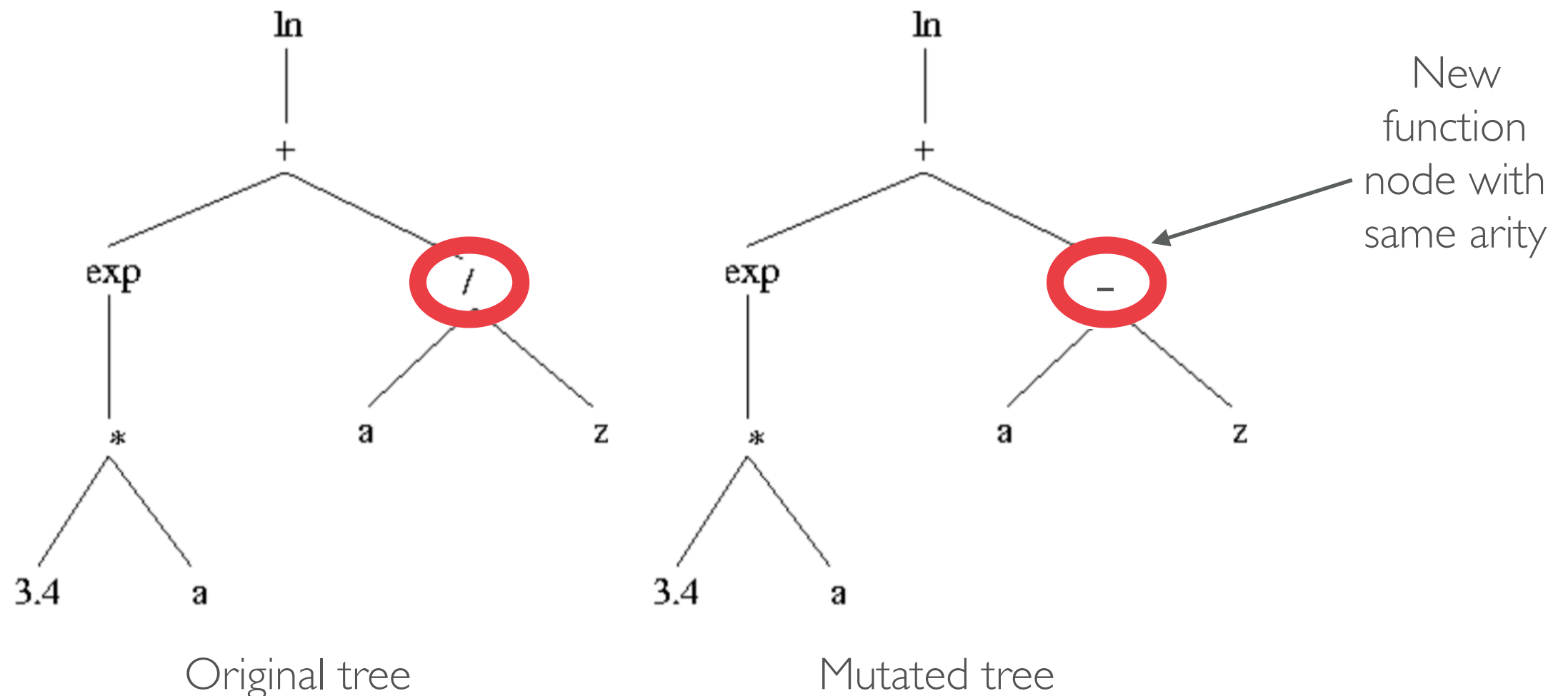
- function node mutation
- terminal node mutation
- growth (expansion) mutation
- truncation mutation
- swapping mutation
- Gaussian mutation

on terminal we can have some value that
i may want to fine tune



GENETIC PROGRAMMING

ALGORITHMIC DETAILS: FUNCTION NODE MUTATION

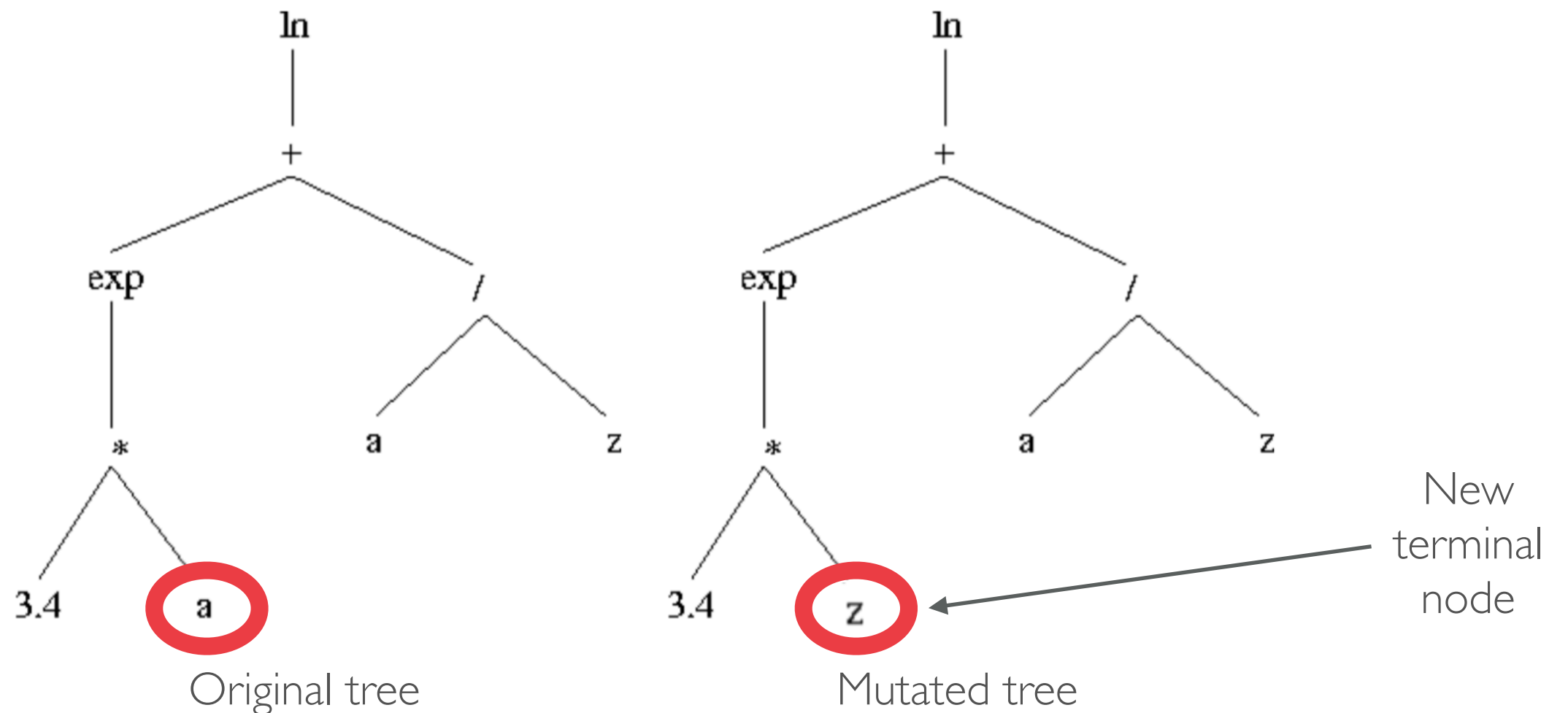


Here we can have problem with the arity of the function that are no longer correct with the function node mutation

Also for example we can have undefined case with division and logarithm so we need a protected division or logarithm (to prevent that the algo get stuck)

GENETIC PROGRAMMING

ALGORITHMIC DETAILS: TERMINAL NODE MUTATION

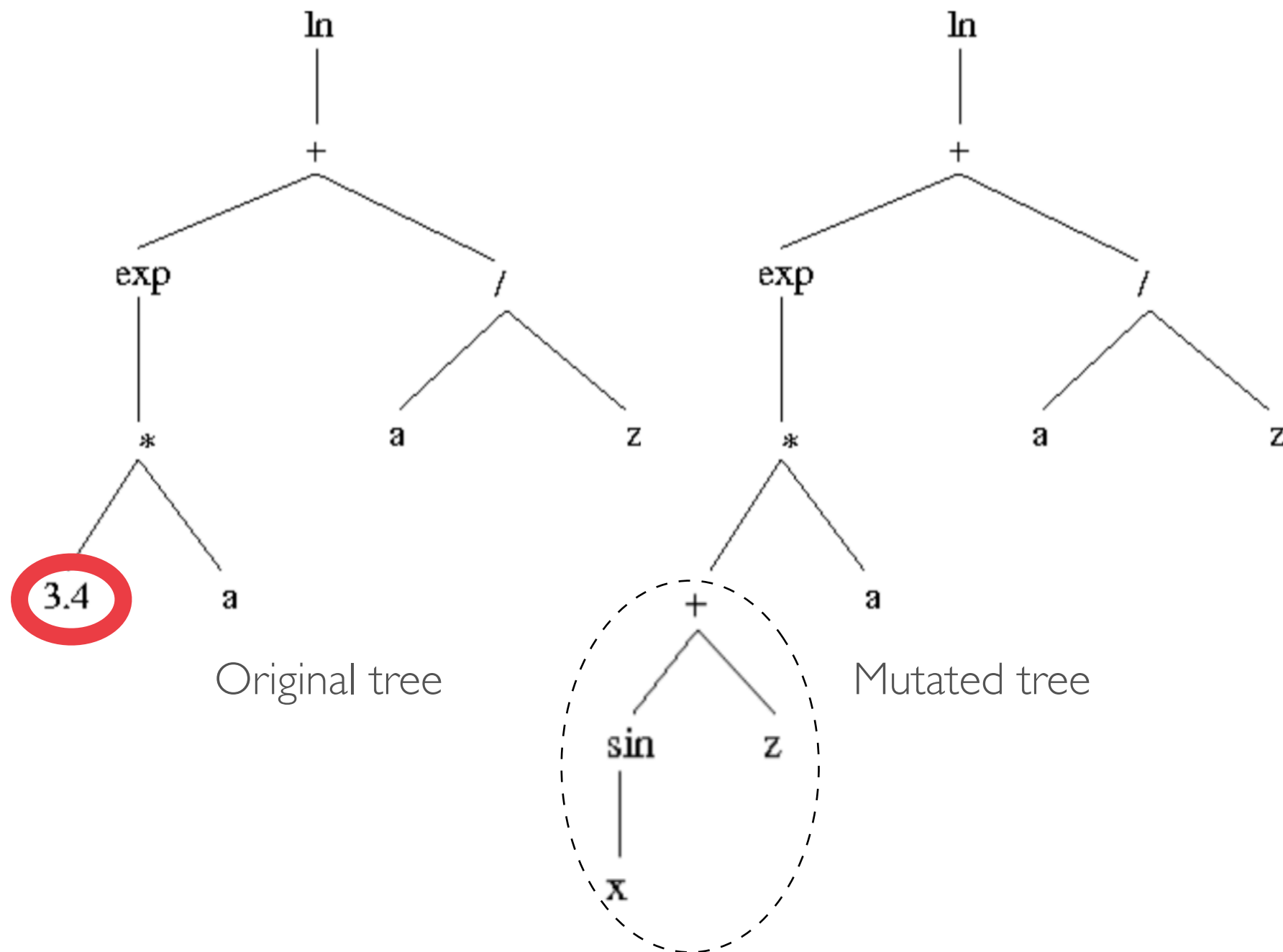


GENETIC PROGRAMMING

ALGORITHMIC DETAILS: GROWTH MUTATION

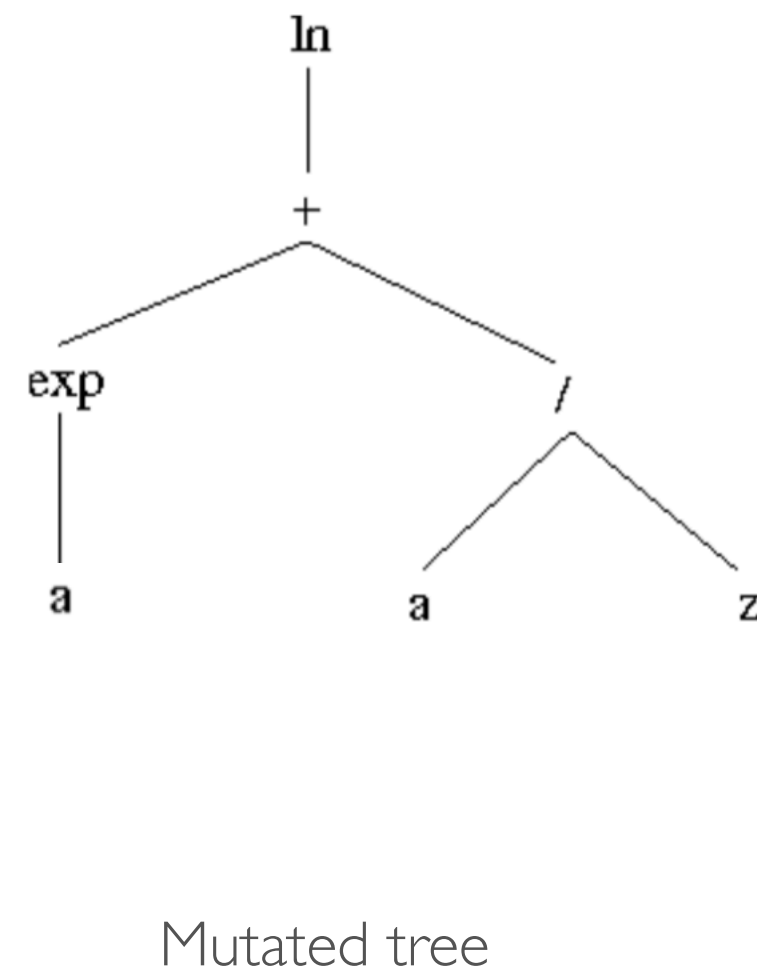
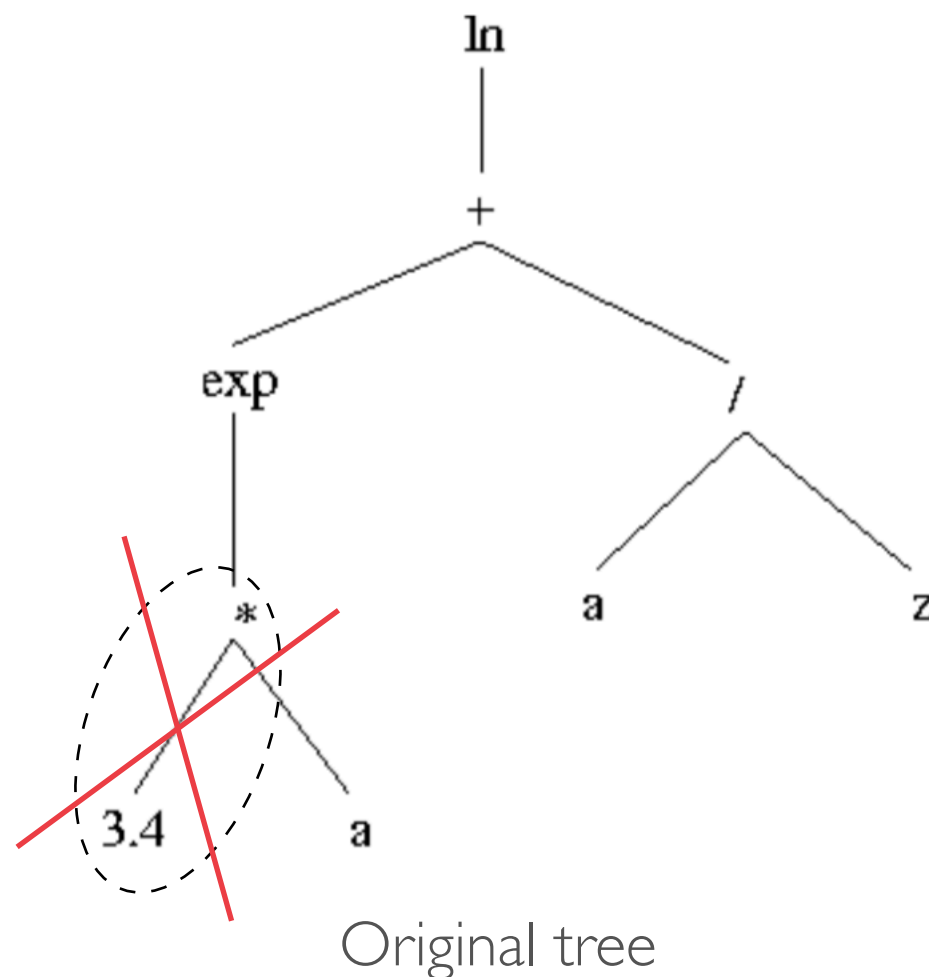
Instead of swaping ; we can add at a specific cutpoint a whole new subbranch (that is generated by ... idk)

It is needed to complexify your tree (add the possibility that the tree grows over time)



GENETIC PROGRAMMING

ALGORITHMIC DETAILS: TRUNCATION MUTATION

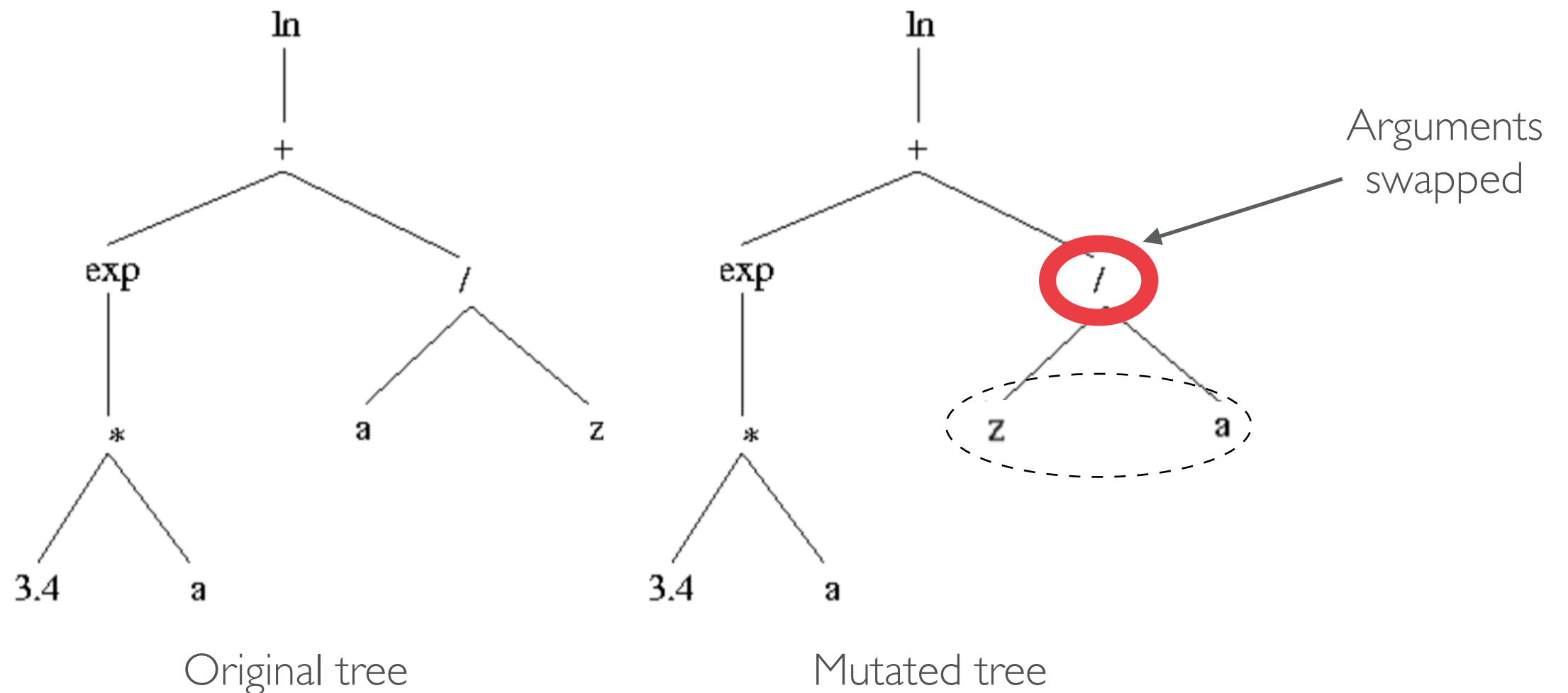


We can prune, you randomly select a cutpoint in your tree and you select a non terminal and we replace it with a terminal node.

It is use to regulate your tree and decomplexify it.

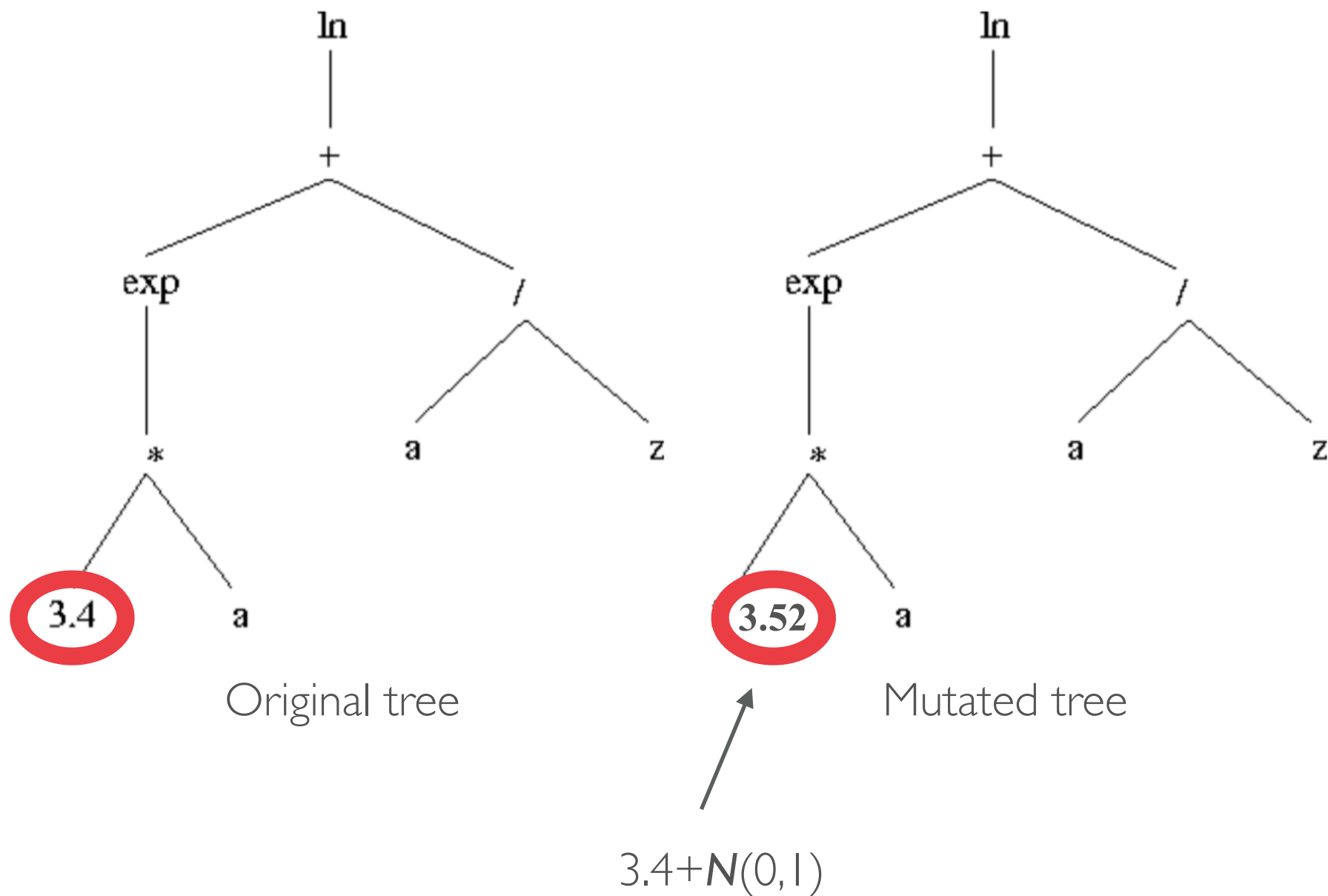
GENETIC PROGRAMMING

ALGORITHMIC DETAILS: SWAPPING MUTATION



GENETIC PROGRAMMING

ALGORITHMIC DETAILS: GAUSSIAN MUTATION



GENETIC PROGRAMMING

ALGORITHMIC DETAILS: STOP CONDITIONS

- Similar to other EAs
 - no. of fitness evaluations
 - no. of generations
 - no. of generations without improvement
 - elapsed time
 - etc.
- Additional criterion for symbolic regression problems:
 - no. of “hits” reached (where “hit” is if $|f(x_i) - y_i| < \epsilon$, e.g. stop when 5000 hits with $\epsilon=0.0001$)
- Multiple criteria can be combined (usually, in OR)



GENETIC PROGRAMMING

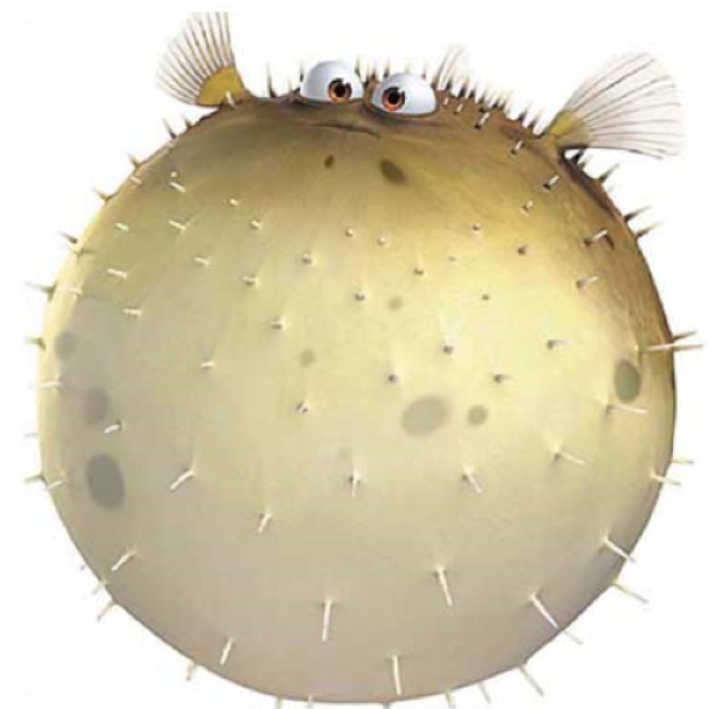
SUCCESSFUL APPLICATIONS

- Robotics
 - automatic generation of gaits for biped robots
 - automatic generation of hand gestures for humanoid robots
 - automatic generation of mouth motions for a speaking robot head
 - automatic generation of a state machine for a behavioral robot (Braitenberg Vehicles)
- Biology
 - transmembrane segment identification problem for proteins
- Quantum Computing
 - automatic design of algorithms and communication protocols
- Electronics and TLC
 - design of of analog circuits (topology and RLC parameters), controllers and antennas (NASA)
- System identification and automatic discovery of physics laws (Eureka)

GENETIC PROGRAMMING

ISSUES IN GP: BLOAT

- Bloat = “survival of the fattest”: literally, i.e. the GP trees in the population increase in size over time, in order to fit better and better the input data (somehow linked to overfitting)
- BUT, too big GP trees might be difficult to handle/understand
- Possible countermeasures:
 - Limit the size of the trees in the initial population
 - Prohibit variation (mutation & crossover) operators that would generate “too big” trees
 - Introduce checks/repair mechanisms in mutations & crossover
 - Pruning (worst trees replaced by branches pruned from best tree)
 - Penalize the fitness of oversized trees (parsimony pressure)
 - Use ad-hoc operators
- Ongoing research and debate about the reasons



GENETIC PROGRAMMING

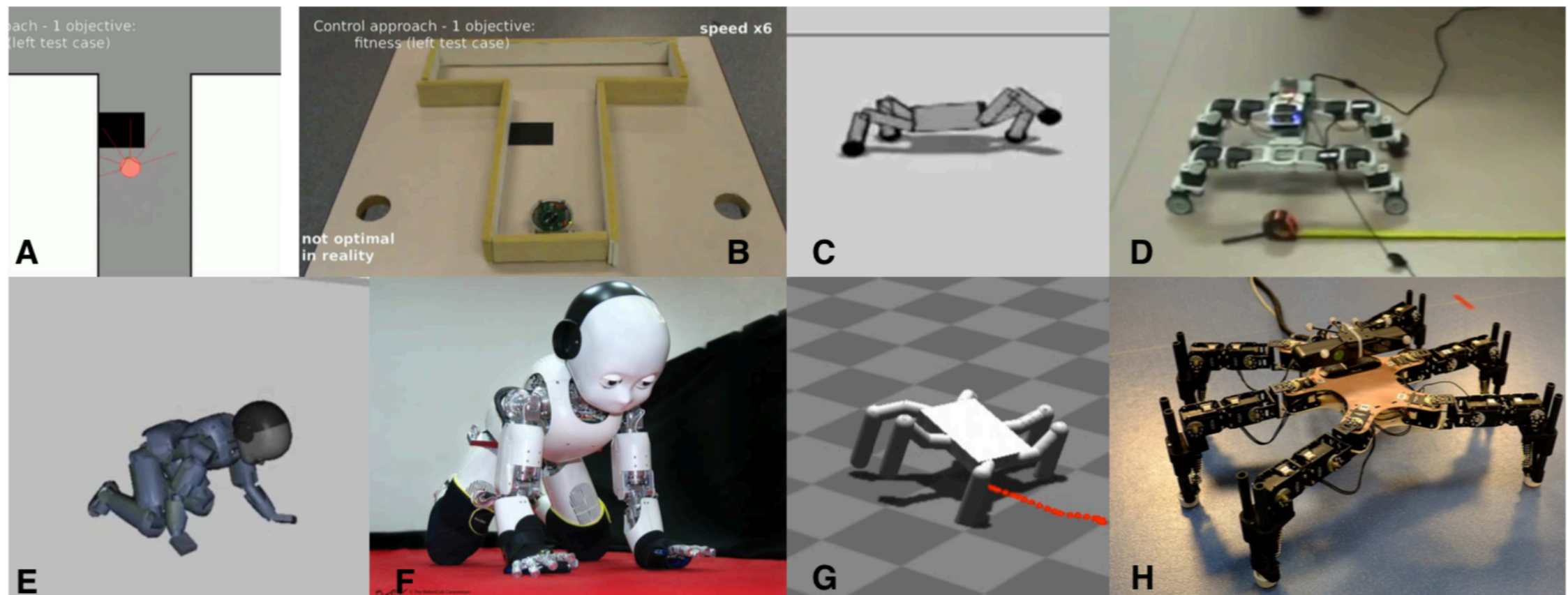
ISSUES IN GP: BLOAT (“BUILDING BLOCK” APPROACH)

- Start with the simplest trees, then let trees grow during the evolutionary process, as needed
- Initial individuals consist of only a root and the first children of that node
- Expand trees when:
 1. the simplicity of the population's individuals can no longer account for the complexity of the problem to be solved
 2. no improvement in the fitness of any of the individuals within the population is observed
- Expansion occurs by adding a randomly generated building block (i.e. a new node) to individuals
- Expansion occurs at a specified expansion probability, p_e
- Helps to reduce the computational complexity of the evolution process, and to produce smaller individuals

GENETIC PROGRAMMING

ISSUES IN GP: TRANSFER TO REALITY

- Trees for data fitting vs. trees (programs) that are “really” executable
- Execution can change the environment → the fitness calculation should account for that
Example: a robot controller
- Fitness calculations mostly by simulation, ranging from expensive to extremely expensive (in time)
- BUT, evolved controllers are often very good!



GENETIC PROGRAMMING

ISSUES IN GP: EFFICIENCY

- GP trees are usually very hard to evolve
 - may require a population of thousands of individuals
 - convergence may be very slow (depends on the problem complexity, and the no. of input data)
 - expensive fitness computation (typically each tree execution is interpreted)
- Possible solutions:
 - Parallel GP (e.g.. multithread GP or GP on GPU), with parallelism at tree or instruction level
 - Compiled GP (trees are precompiled with compiling optimizations before execution)



```
class F {  
  int fact(int n) {  
    // offsets 0-1  
    int ans = 1;  
  
    // offsets 2-3  
    if (n > 0)  
      // offsets 6-15  
      ans = n *  
        fact(n-1);  
  
    // offsets 16-17  
    return ans;  
  }  
}
```



```
0 iconst_1  
1 istore_2  
2 iload_1  
3 ifle 16  
6 iload_1  
7 aload_0  
8 iload_1  
9 iconst_1  
10 isub  
11 invokevirtual #2  
14 imul  
15 istore_2  
16 iload_2  
17 ireturn
```

GENETIC PROGRAMMING

DISCUSSION

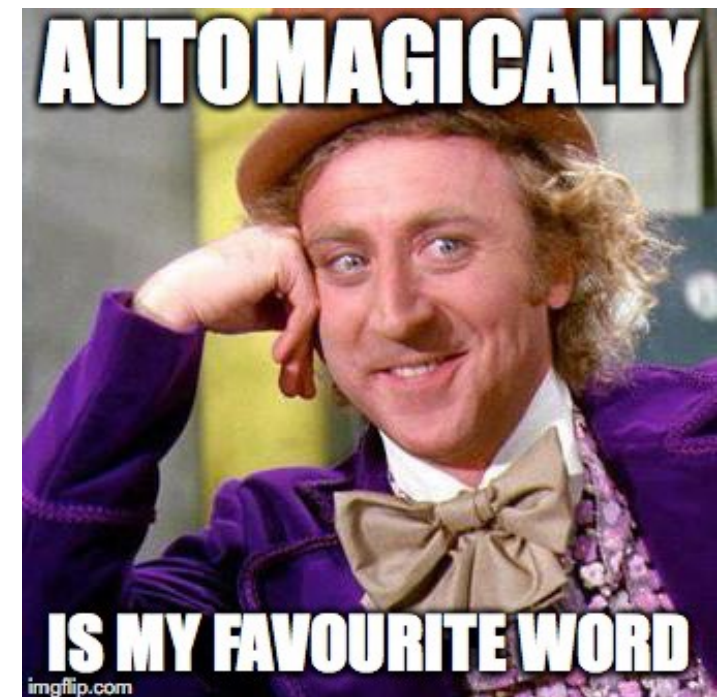
Question:

What is, in the end, GP?

Answers:

- The art of evolving computer programs
- Means to “automagically” programming computers
- GA with another representation

Automagically adv [from automatic and magic, computer jargon, mid 80s] in an $\rightarrow$ *automagical* manner. 1. automatically, but in a way that the speaker doesn't want to explain, typically because it is too complicated, or too ugly, or too trivial. 2. sarcastic, ironically implying an impossible process.



Questions?